Search in course

Search

English

B

1. CS 498: Cloud Computing Applications
2. Week 16
3. MP5 Full Stack (Optional - Beta Testing)

PreviousNext

Programming Assignment: MP5 Full Stack (Optional - Beta Testing)

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

In this Machine Problem (MP), you'll create a full-stack Event Board using real-world AWS tools. You'll deploy a backend server and a frontend website using Platform as a Service (PaaS) tools like S3, Elastic Beanstalk, RDS, CloudFront, CodePipeline, and more.

This project is designed to simulate what happens in real-world cloud deployments. You'll use automated pipelines to build and deploy, and you'll connect a backend API to a database, just like production applications.

2. Requirements

Before you begin, make sure you have:

- A valid AWS account
- Two private GitHub repo, one for your Frontend and one for your Backend

- Familiarity with:
    - Python & Flask (backend)
    - Basic React (frontend)
    - AWS Services: S3, CloudFront, Elastic Beanstalk, Amazon RDS, CodeBuild, CodePipeline

Note: We suggest you create all of the services in the zone us-east-1 to prevent any unexpected issues from the autograder.

3. Procedure

3.1 Setup

The base code for the frontend and backend of the assignment are available in a public GitHub repository: https://github.com/UIUC-CS498-Cloud/MP5_FullStack_Template

What you need to do first:

1. Create two new private GitHub repositories under your own account, one for your Frontend Template, second for your Backend Template
2. Clone the public template repo to your local machine
3. Push the Frontend directory contents to your newly created private Frontend repo

1

2

3

4

5

6

7

8

9

git clone https://github.com/UIUC-CS498-Cloud/MP5_FullStack_Template

cd MP5_FullStack_Template/Frontend

git init

git remote add origin {your-github-frontend-repo-url}

git add .

git commit -m "Frontend only"

git branch -M main

git push -u origin main

Please ensure that the root of your frontend repository directory looks like:

- public/
- src/
- buildspec.yml
- package.json

4. Push the Backend Directory contents to your newly created private Backend repo

1

2

3

4

5

6

7

8

git clone https://github.com/UIUC-CS498-Cloud/MP5_FullStack_Template

cd MP5_FullStack_Template/Backend

git init

git remote add origin {your-github-backend-repo-url}

git add .

git commit -m "Backend only"

git branch -M main

git push -u origin main

Please ensure that the root of your backend repository directory looks like:

- tests/
- application.py
- buildspec.yml
- requirements.txt

This is required as your private repos will be used by AWS CodePipeline as the source for deployment.

3.2 S3

We will use S3 to deploy our frontend build files due to its low storage costs and it acts works well with CDNs such as CloudFront. We will also use this bucket to store the CICD logs.

- Create a private S3 bucket
- Inside the bucket, make two folders:
    - build : where your React app will be deployed
    - frontend-logs : where CodeBuild will store frontend logs
    - backend-logs : where CodeBuild will store backend logs
- Disable static website hosting

3.3 CI/CD with CodePipeline for Frontend

CI/CD is an important aspect in real world staging and production deployments to make sure all the new changes made to an exisiting codebase is continuously integrated and continuously deployed to the outside world. We will create two pipelines during this MP in total. This step will help you create an introductory pipeline for your frontend codebase.

Set up your first pipeline:

1. Go to the **CodePipeline dashboard**
2. Click “Create pipeline” and choose "Build Custom Pipeline"

3. Give it a name and choose "Queued" execution mode
4. Let AWS create a new **service role** for you

Source Stage:

For this we will use the most common source stage for CICD pipelines: Github

- Choose **GitHub (via GitHub App)**, recommended
- Follow these instructions until Step 2 under "Create a connection to GitHub (console)" to setup a github connection: https://docs.aws.amazon.com/codepipeline/latest/userguide/connections-github.html
- Select both of your backend and frontend repositories under the Github configuration for AWS connector
- Make sure the GitHub App has:
    - Read access to deployments, issues, metadata
    - Read/write access to admin, code, pull requests, commit status

Once the connection is saved, select your frontend repo in the CodePipeline console and branch (main) and proceed.

Build Stage:

For the build stage we will use a buildspec.yml file provided with your template. Amazon CodeBuild automatically looks for this file inside your repo and follows the instructions to complete this step. Hence we will go through how to set up CodeBuild with CodePipeline.

- Under Other Build Providers, choose **CodeBuild** as your option
- Create a new build project & give it a name
- To fall within the AWS Free Tier, under Environment -> Additional Configuation, please use " 3 GB memory, 2 vCPUs " compute.
- Under the Buildspec section enter the name of frontend buildspec file (ie **buildspec.yml**) provided in the starter code
- For logs, select your S3 bucket and enter "frontend-logs" (without slashes) as the prefix. This will produce all the build logs within that path of your bucket which you will share to the autograder using a pre-signed URL.

Test Stage:

We'll **skip** this stage for Frontend.

Deploy Stage:

- Choose **S3** as the deploy provider
- Select your previously created S3 bucket
- Use "build" (no slash) as the object key
- Enable **"Extract file before deploy"**

Once deployed, CodePipeline will automatically update your frontend whenever new changes are pushed to the repo.

Debugging Resources:

https://docs.aws.amazon.com/codepipeline/latest/userguide/connections-github.html

https://docs.aws.amazon.com/codepipeline/latest/userguide/pipelines-create.html\

3.4 CloudFront

To serve the frontend files to the public, most commonly a Content Delivery Network (CDN) is used for lower latency. So we will create a public-facing CDN - AWS CloudFront to serve your frontend while keeping the S3 bucket private.

Setup:

- Origin Domain: {bucket-name}.s3.{region}.amazonaws.com
- Origin Path: /build
- Enable **Origin Access Control** (OAC)
- Under Web Application Firewall-> Select Do not enable security protections.
- Set **Default Root Object** to index.html
- After creating, update the S3 bucket policy, AWS will give you an automatic generated policy similar to:

14

15

16

```
17
18
19
20
21
11
12
13
8
9
10
6
7
3
4
5
1
2
            "StringEquals": {
               "AWS:SourceArn": "{your-cloudfront-distribution-ARN}"
            }
         }
      }
   ]
}
```

```
        "Action": "s3:GetObject",
        "Resource": "arn:aws:s3:::{your-bucket-name}/*",
        "Condition": {
        "Principal": {
          "Service": "cloudfront.amazonaws.com"
        },
        "Sid": "AllowCloudFrontServicePrincipal",
        "Effect": "Allow",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
{
  "Version": "2008-10-17",
```

Test the CloudFront domain. If you see “Access Denied,” revisit the OAC or S3 policy. If domain/index.html works but domain/ doesn’t, then the default root object was not set.

3.5 Deploying Backend with Beanstalk + RDS

Now that you have set up your frontend serving capability correctly along with a frontend pipeline we will move onto the backend deployment. To deploy our backend, AWS has this wonderful platform as a service Elastic Beanstalk that handles automatic versioning, scaling, security and RDS integration without having to manually connect everything.

Deploying to Beanstalk:

1. Go to Elastic Beanstalk dashboard and create a new application
2. Select platform: **Python**
3. Keep the sample code, we will use a second backend CICD pipeline to deploy your custom backend logic and see a CICD run in action.
4. IAM Setup in next step:
    - Let AWS create a **service role** for you

- Create an **instance profile** before submitting to the Beanstalk with policies:
    - AWSElasticBeanstalkEnhancedHealth
    - AWSElasticBeanstalkWebTier
    - AmazonRDSFullAccess
    - AWSElasticBeanstalkMulticontainerDocker

5. Next, under "Set up networking, database, and tags", enable RDS and keep default settings (MySQL)
6. Create a username/password, you'll use this later

In all the next steps, keep the default configurations and then hit submit. Wait until Beanstalk shows “Healthy.” If errors appear, they’re likely caused by incorrect role or instance profile settings.

3.6 Add Environment Variables in Beanstalk

After you have created an automatic platform, you need to configure a few environment variables so that your backend template/REST API can communicate with your DB using os environment variables.

Note: Do not hardcode database credentials that is not a good programming practice, always use environment variables.

Suggested flow: Add the given variables by going into Elastic Beanstalk Environment -> Configuration -> Updates, Monitoring & Logging -> Edit ->

Add Env Properties as a plaintext where the Keys are on the left, for values look at the instructions on the right.

Key	**Value**
DB_HOST	This is your RDS endpoint, don't include the port
DB_USER	Put the user name you created in the previous step
DB_PASSWORD	Put the password you created in the previous step
DB_NAME	Check the RDS console for your database instance. Inside there look within the configuration section for a database name, most likely it will be “ebdb”

3.7 CI/CD with CodePipeline for Backend

Set up your second pipeline for Backend:

1. Go to the **CodePipeline dashboard**
2. Click “Create pipeline” and choose "Build Custom Pipeline"
3. Give it a name and choose "Queued" execution mode
4. Let AWS create a new **service role** for you

Source Stage:

Reuse the previous Github connection used while setting up the pipeline for Frontend. After that select your Backend repo and branch (main) and proceed.

Build Stage:

We’ll **skip** this stage for Backend.

Test Stage:

We have created few unit tests to automatically run testing during the build process. They are stored under /tests directory in your /Backend template.

- Under Test Providers, choose **CodeBuild** as your option
- Create a new project & give it a name
- To fall within the AWS Free Tier, under Environment -> Additional Configuation, please use " 3 GB memory, 2 vCPUs " compute.
- Under the Buildspec section enter the name of backend buildspec file (ie **buildspec.yml**) provided in the starter code
- For logs, select your S3 bucket and enter "backend-logs" (without slashes) as the prefix. This will produce all the build logs within that path of your bucket which you will share to the autograder using a pre-signed URL.

Deploy Stage:

- Choose **Elastic Beanstalk** as the deploy provider
- Select your previously created Beanstalk application and environment

5. Lastly go to IAM and search for the automatic Role AWS created for this pipeline and add " AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy " to the role. It lets you create a new version of your python code deployment to Beanstalk.

Once deployed, CodePipeline will automatically update your backend code to Beanstalk whenever new changes are pushed to the repo.

3.8 Backend Setup (Flask + DB)

This part of the fullstack application handles all the computational logic and connects the Database with your frontend. We've provided you with a backend template. You only need to implement two functions:

1. insert_data_into_db(payload)
    - Use PyMySQL to insert title, description, date, location into the events table.
2. fetch_data_from_db()
    - Select all rows and cols and return them in **ascending order of date**

Our payload looks like:

```
1
2
3
4
5
6
{
   "title": "Music Festival",
  "date": "2025-06-25",
  "description": "UIUC Music Festival",
  "location": "Champaign"
}
```

"id" is an auto-incrementing attribute.

The frontend uses the implemented endpoints in this template to hit a particular function.

The frontend is currently not wired to call your backend, in the next semester we are planning to give an optional challenge (yet exciting to watch everything come together in one place) to implement a tanstack query in the last section. You can read more about Tanstack over here: https://tanstack.com/query/latest

4. Our Test Cases

#	TC	Description
1	Frontend Content Test	We check if you successfully deployed a cloudfront CDN distribution a is correctly returning the expected frontend content.
2	Backend Health Test	We check if you successfully deployed an Elastic Beanstalk Backend a its health.
3	Backend + DB POST Integration Test	We make a POST request to your Backend with a payload that will inse some data inside your Database.
4	Backend + DB GET Integration Test	We make a GET request to your Backend to confirm that your GET meth work and in turn the autograder payload is present in the RDS Databas
5	CICD Frontend Log Test	We check your Frontend S3 logs to verify your Frontend CICD pipeline.
6	CICD Backend Log Test	We check your Backend S3 logs to verify your Backend CICD pipeline.

5. Checklist

To let our autograder access your bucket objects without you having to configure any IAM policies or making it public, this is a good point to introduce the concept of pre-signed URL. It enables you to share particular objects with anyone for a set period of time without having it set to public or searchable. Make sure you submit the presigned URL so our autograder can access your most recent log. Look into this guide https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html and select the most recent log available under /frontend-logs & /backend-logs directory with a successful CodePipeline run, then select that object to copy the presigned URL for a set amount of time (for example: 3 hours, if you intend to run the autograder beyond that make sure to generate a new URL) and then submit this URL in the submission.py.

Before submitting the assignment, it would be great to check if you have successfully configured the following services:

- One private S3 that contains frontend build and the logs
- Two presigned URL S3 for latest successful CodeBuild frontend and backend logs respectively
- Cloudfront Domain
- Beanstalk URL
- Active connection between Beanstalk and DB

6. Submission

Add the necessary info in the submission.py file attached below in the payload section. If all the test cases pass, you will recieve a message: All Test Cases Passed. If not, you will see the failed test cases and debug info.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

```
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
import requests
```

```
import json

YOUR_EMAIL = "" # put your Coursera email
YOUR_SECRET = "" # put your secret token from Coursera

# 1. Frontend Deployment (CloudFront URL)
CLOUDFRONT_URL = "" # put your CloudFront domain

# 2. Backend Deployment (Elastic Beanstalk URL)
BEANSTALK_URL = ""  # put your Beanstalk backend URL

# 3. CICD (Frontend S3 Log URL) - a presigned URL to access the Frontend CodeBuild logs.
FRONTEND_S3_LOG_URL = "" # put your presigned S3 Frontend log URL

# 4. CICD (Backend S3 Log URL) - a presigned URL to access the Backend CodeBuild logs.
BACKEND_S3_LOG_URL = "" # put your presigned S3 Backend log URL

#Autograder URL
LAMBDA_URL = "https://ttrejw3rxf.execute-api.us-east-1.amazonaws.com/default/mp5_fullstack"


input_data = {
  'cloudfront_url': CLOUDFRONT_URL,
  'beanstalk_url': BEANSTALK_URL,
  'frontend_s3_log_url': FRONTEND_S3_LOG_URL,
```

```
    'backend_s3_log_url': BACKEND_S3_LOG_URL,
    'submitterEmail': YOUR_EMAIL,
    'secret': YOUR_SECRET
}

payload = {
    "input": json.dumps(input_data),
}

try:
    response = requests.post(LAMBDA_URL, data=json.dumps(payload))
    print(response.status_code, response.reason)
    print(response.text)
except Exception as e:
```

7. FAQs

TBD

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue